

WEEK4 PDF DOCUMENT SUMMARIZER (RAG PROJECT)



**LEARN
WITH
SARVESH**

COURSES. CLARITY. CONFIDENCE.



12 Week Agenda

Week 1: Introduction to LangChain & AI Workflows

Week 2: Working with Prompts, Chains & Memory

Week 3: AI Resume Analyzer

Week 4: PDF Document Summarizer (RAG Project)

Week 5: AI Email Assistant

Week 6: YouTube Transcript Summarizer



12 Week Agenda

Week 7: Customer Support Chatbot

Week 8: Website Q&A Chatbot

Week 9: Job Recommendation Agent

Week 10: Meeting Notes Generator

Week 11: Travel Itinerary Planner Agent

Week 12: Local Knowledge Assistant



Week 4

PDF DOCUMENT SUMMARIZER – RAG PROJECT



Week 4: Learning Objectives

Understand what RAG (Retrieval-Augmented Generation) is

Load and process PDF documents using LangChain loaders

Split large documents into meaningful chunks

Generate embeddings and store them in a vector database (FAISS)

Retrieve relevant document context for a user query





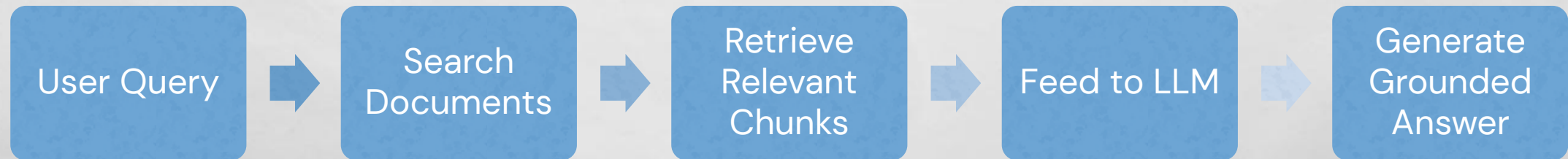
Prerequisites Check

- ☒ OPENAI API KEY (GET FROM PLATFORM.OPENAI.COM)
- ☒ PYTHON 3.8+ ENVIRONMENT
- ☒ SAMPLE PDF DOCUMENT
- ☒ REQUIRED LIBRARIES INSTALLED



What is RAG?

- **PROBLEM:** LLMS DON'T KNOW ABOUT YOUR SPECIFIC DOCUMENTS
- **SOLUTION:** RETRIEVE RELEVANT CONTEXT + GENERATE ANSWER



Loading PDFs

PyPDFLoader Capabilities:

Extracts text from each page

Preserves page numbers
(metadata)

Handles various PDF formats

Each page = separate Document
object





Token Limits: LLMs have context windows (4K–128K tokens)



Better Retrieval: Smaller chunks = more precise matches



Cost Efficiency: Send only relevant text, not entire PDF



Embedding Quality: Focused chunks embed better

Why Chunk Documents?



RecursiveCharacterTextSplitter

[CHUNK 1: CHARS 1-1000]

[CHUNK 2: CHARS 800-1800]

[CHUNK 3: CHARS 1600-2600]



What Are Embeddings?

Convert text → numerical vectors (arrays of numbers)

Capture **semantic meaning**, not just keywords

Similar texts = similar vectors



What is FAISS?

- FACEBOOK AI SIMILARITY SEARCH
- BLAZING-FAST VECTOR SEARCH LIBRARY
- HANDLES MILLIONS OF VECTORS EFFICIENTLY
- OPEN-SOURCE AND PRODUCTION-READY

Without FAISS: Compare query to ALL chunks (slow ❌)

With FAISS: Smart indexing = instant search (fast ✅)



How FAISS Works

Building the Index:

- Take all chunk embeddings
- Organize into efficient search structure
- Store index in memory (or disk)

Searching:

- Convert query to embedding
- Find k nearest neighbors in milliseconds
- Return most similar chunks



RAG Pipeline

User Query

Retriever (Find top 3) ← Search vector store

Format Context ← Combine chunks

Prompt Template ← Add instructions

LLM (GPT-3.5) ← Generate answer

Final Answer



RAG Components

1. Retriever:

- Searches vector store
- Returns k=3 most relevant chunks

2. Prompt Template:

- System instructions
- Context placeholder
- User question

3. LLM:

- GPT-3.5-turbo (fast & cheap)
- Temperature=0 (factual answers)

4. Chain:

- Connects all components with LCEL



Cost Breakdown

EXAMPLE: 100-PAGE PDF

- PDF LOADING: FREE
- CHUNKING: FREE
- EMBEDDINGS: $\sim 500 \text{ CHUNKS} \times \$0.02/1\text{M} = \text{\$0.01}$
- QUERIES (100 QUESTIONS): $\sim \$0.05$
- **TOTAL: $\sim \$0.06$ PER DOCUMENT**

💡 ONE-TIME EMBEDDING COST + PER-QUERY LLM COST



Production Checklist



Environment variables for API keys



Error handling and logging



Rate limiting for API calls



Vector store persistence



User authentication



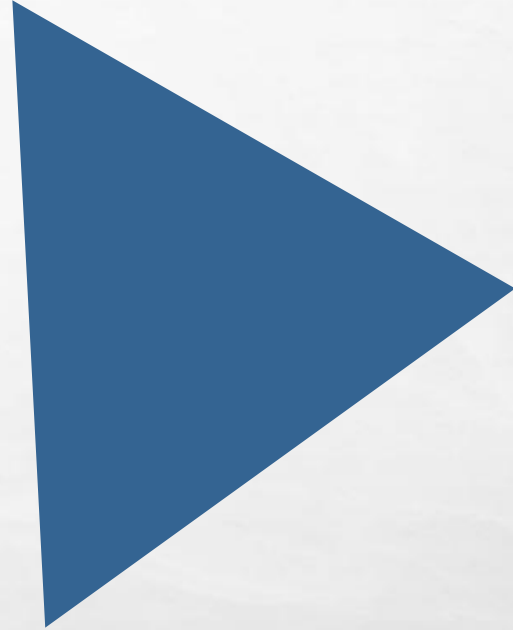
Response caching



Monitoring with LangSmith



CODE WALKTHROUGH



THANK YOU!



**LEARN
WITH
SARVESH**

COURSES. CLARITY. CONFIDENCE.

